INTER.act - Progress Report: Sneh Dungrani

Contributions up to Review Party 1 (08 May 2026)

Project: INTER.act - Autonomous Multi-Agent Robotic Coordination via Agentic AI
Contributor: Sneh Dungrani
Branch: `dev-sneh`
Reporting Period: 30 March 2026 – 08 May 2026
Review Party 1 Date: 08 May 2026

---

1. Foundational work completed before RP1

1.1 Technology stack decision

I pushed for Python and Flask as the full-stack backbone, and we dropped Next.js early. Every arm script, agent logic module, and state manager was already Python - adding a Node layer between the scripts and the browser would have created a Python → Node → React boundary on every state update. For a 6-week prototype, that was complexity with no payoff. Flask with Flask-SocketIO handled both the HTML frontend and the WebSocket layer from the same process.

1.2 Architecture agreed on

The container topology we locked in before the first sprint switch:

```
frontend (nginx:8080) → backend (Flask:5000) ← demo-runner (Flask:6000)
                                ↑         ↑
                   arm-1..N (Flask:8001..)   veh-* (Flask:8010..)
```

A few decisions that shaped everything downstream:

- The demo-runner got its own container. Running orchestration inside the main Flask process starved the Werkzeug thread pool and blocked SocketIO - we hit this early and separated them.
- `pipeline.yaml` became the single source of truth. Topology changes go through config, not hand-edited compose files.
- Threading-only async mode, no eventlet. The SocketIO/eventlet deadlock class is well-documented and not worth touching.
- The browser receives state via SocketIO push. It never polls.

---

2. My technical contributions

2.1 Repository setup and branch workflow (02 April 2026)

I created the GitHub repository and set up the branch structure the team used for the whole sprint. I split the repo into `/app`, `/scripts`, `/docker`, `/tests`, `/docs`, and `/planning`, wrote the README with quick-start and

documentation map, and defined the per-member branch convention - `dev-sneh`, `dev-haris`, `dev-purvil` - so parallel development lines didn't keep colliding. I also published a system documentation PDF on day one so the team had a shared architectural reference before anyone wrote a line of code.

2.2 Scene configuration and UI rendering refactor (11 April 2026)

The canvas had its layout values scattered across constants in multiple files. I consolidated them into a single configuration object - static layout sent once on connect, dynamic arm state pushed on every angle change. That separation became the pattern everything else in the frontend followed.

2.3 Ollama and Qwen integration into Docker (15 April 2026)

I added the Ollama service to the Docker Compose stack and wired up health checks that gate arm container startup behind Ollama availability. I picked `qwen2.5:0.5b` - small enough to run on CPU during development while still being an actual LLM in the loop. Updated the README so everyone could replicate the environment without asking me. That was the first time a local LLM was available to the pipeline in a reproducible, containerised form, and the AI brain work on `dev-haris` built directly on top of it.

2.4 Backend SSOT hardening and health monitoring (21 April 2026)

I split configuration reload from runtime restart into two independent endpoints. Before that, any config change needed a full pipeline restart, which lost in-flight object state mid-demo. After, topology updates applied cleanly to a running stack.

I also fixed a health monitor bug where draining arms were being flipped back to recovering and then idle, bypassing the drain-then-stop lifecycle entirely. That guard - restricting `recover()` to `failed` or `recovering` states only - is now a documented invariant in `CLAUDE.md`. On top of that I enhanced the monitor to emit structured socket events on state transitions so the frontend could show failures and recoveries in real time.

2.5 Frontend multi-asset simulation and graph-driven layout (21 April 2026)

The canvas was hardcoded for three arms at fixed pixel positions. I replaced that with a layout engine that reads the asset topology at page load and positions everything from the pipeline graph. Handoff zones are filtered so only arm-to-vehicle boundaries get drawn as rectangles - arm-to-arm transfer points within the same stage are left out. Object ownership events propagate correctly across asset type boundaries. After this change, adding or removing an arm in `pipeline.yaml` required zero JavaScript edits.

2.6 Asset controller architecture (27 April 2026)

This was the biggest chunk of engineering I did before RP1. I designed and built the abstraction layer that let vehicles participate in the pipeline on equal footing with arms.

I started with a `Module` base class that defines the full lifecycle interface: `start`, `stop`, `fail`, `recover`, `drain`. Every asset type implements the same contract. On top of that I built `AssetController` and a concrete `VehicleController`, with transfer-point logic using a condition variable for atomic, polling-free object handoff between containers. The previous event/lock approach had a signal-loss race; the condition variable closed it.

I also wrote the single configuration layer that validates the full topology at load time - checking for dependency issues and unknown asset types - and is consumed identically by the backend, demo-runner, and

compose generation. The compose generation script I wrote derives a complete `docker-compose.yml` from `pipeline.yaml` automatically, so the compose file is never hand-edited. Vehicle containers got their own microservice entrypoint running the controller loop and exposing health and transfer HTTP endpoints.

I wrote tests covering controller state-machine transitions, asset graph validation, and module contract compliance under multi-vehicle topologies. When Haris merged `dev-sneh` into the integration branch on 03 May, the AI brain generalised across arms and assets cleanly because the topology was already config-driven and the asset state contract was uniform.

2.7 Flow control, recovery, and ownership rendering (28 April 2026)

I introduced backpressure using a condition variable so arms and containers block when the downstream slot is occupied. Without it, objects could be pushed into a full zone. I fixed object ownership rendering so the canvas positioned objects correctly during vehicle transit - before the fix, objects jumped positions on handover. I also made compose port assignment deterministic from pipeline position, which stopped port collisions when the pipeline grew beyond three arms.

2.8 Dynamic arm count and terminology standardisation (29 April 2026)

I removed all hardcoded arm-count references and replaced them with a runtime lookup from the configuration layer. The backend and frontend now adapt to any number of arms declared in the pipeline file without code changes. I also renamed "handoff" to "handover" across the codebase - Python files, JavaScript modules, config files, documentation - roughly thirty files in total.

2.9 Conveyor subsystem removal (10 May 2026)

I removed the conveyor-belt subsystem from the backend, frontend, tests, and configuration. Rather than silently ignoring an unsupported type, I added validation errors at both config-load time and compose-generation time. Any pipeline configuration declaring a conveyor type now raises an error immediately. All 132 automated tests pass after the removal.

---

3. GitHub management

I ran repository management for the team across the RP1 sprint. I created the repo and defined the branch workflow. I maintained the `.gitignore` across multiple passes - adding artefacts, generated output, cache files, and virtual environment directories as they came up. I resolved `.gitignore` merge conflicts when multiple branches updated it at the same time. I also managed the `dev-sneh` branch through the integration merge on 03 May, where the AI brain layer came in from `dev-haris` and merged cleanly against the pipeline infrastructure.

---

4. RP1 presentation contribution

I wrote the sprint plan document that structured the six-week sprint into epics, weekly checkpoints, and acceptance criteria. That became the backbone of the RP1 narrative. I produced the system documentation that the team used as the technical foundation for presentation content. I committed a stable pipeline baseline ahead of the first sprint review with a clear note on what remained open - dynamic arm failure testing - which kept the team's expectations calibrated going into RP1. I also contributed a standalone lifecycle

diagram showing the arm and asset state machine, which we used during RP1 to walk through failure propagation behaviour.

---

5. Summary

Area	What I did	Why it mattered
Repository setup	Created GitHub repo, defined branch workflow, wrote README	Team could start immediately; no coordination overhead
Docker stack	Compose architecture, Ollama integration, health checks	One-command reproducible environment
Backend infrastructure	Module abstraction, asset controller hierarchy, config layer, compose generation	Config-driven topology; AI branch merged cleanly against it
Health monitoring	Draining guard, health socket events, recovery lifecycle fix	Eliminated false recovery transitions
Frontend rendering	Graph-driven layout engine, multi-asset object ownership, handoff zone filter	Canvas adapts to any topology without JavaScript changes
Flow control	Transfer-point backpressure, compose port sync	No object collision under concurrent transfer
Config SSOT	Decoupled config reload from runtime restart	Topology changes apply without disrupting demos
Codebase quality	Dynamic arm count, terminology rename, conveyor removal	132 passing tests; invalid config fails loudly
GitHub management	Branch workflow, gitignore, merge coordination	Clean parallel development across the team
RP1 presentation	Sprint plan, lifecycle diagram, system documentation	Technical content for the RP1 review

---